

HiCE object branch — change note

Delta against the implementation reference: replace $\sigma(\max(H))$ gate with a supervised presence gate; switch fusion to zero-initialized residual. Everything not listed here stays as in the reference PDF. July 2026.

1. What changes and why (summary)

The previous plan gated the pooled object feature with $\sigma(\max(H))$. That conflates two different questions: the heatmap should answer *where to pool*; a trust signal should answer *whether to use the pooled feature at all*. Using one uncalibrated logit for both is brittle: a wrong sharp peak yields gate ≈ 1 and injects a confident wrong feature; small logits early in training suppress the object path exactly when it needs gradient (chicken-and-egg); and failures cannot be attributed to location vs confidence when debugging.

Fix: keep the heatmap purely as spatial attention; add a tiny supervised **presence head** `q_obj` that predicts whether a usable candidate object exists in the current crop, with its target derived for free from the rendered heatmap grid. Additionally, switch fusion from `Linear(concat(f_global, f_obj))` to a **zero-initialized residual**: `f_out = f_global +  $\gamma$  · proj(f_obj)`. The residual form is not an ablation variant — it is the fusion in every configuration, since step-0 equivalence to vanilla HiCE is a safety property no run should give up.

2. Forward pass: old vs new

OLD (reference PDF):

```
H      = obj_heatmap_head(F)                # (B*L, 1, 7, 7)
A      = softmax(H.flatten(2), dim=-1)
f_obj  = (F.flatten(2) * A).sum(-1)          # (B*L, D)
f_obj  = sigmoid(H.amax(dim=(2,3))) * f_obj  # gate = sigma(max(H))  <-- REMOVE
im_feat = Linear(cat(f_global, f_obj))      # concat fusion    <-- REPLACE
```

NEW:

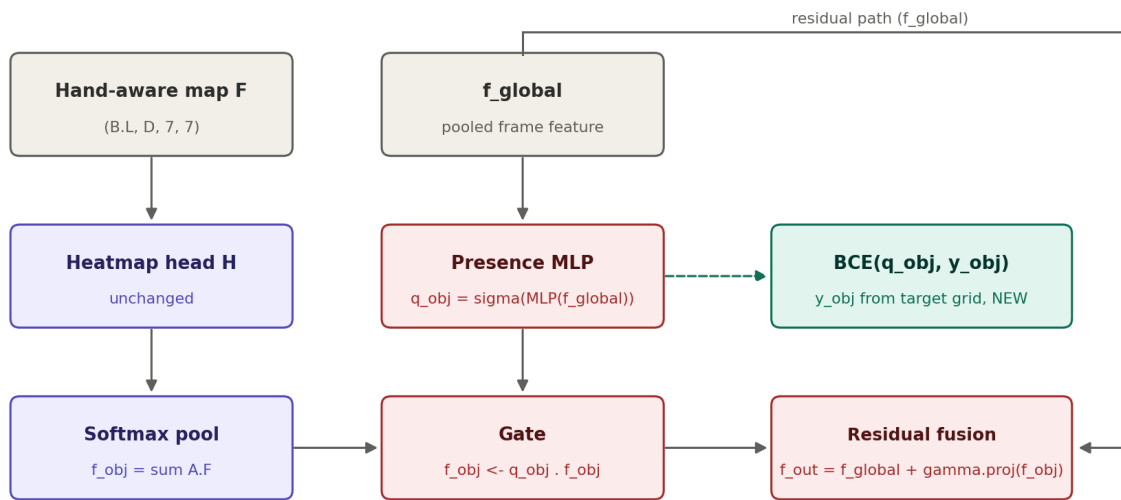
```
# F: (B*L, D, 7, 7) post-FFN hand-aware map; f_global: (B*L, D) existing pooled feature
H      = self._obj_head(F)                  # (B*L, 1, 7, 7) -- unchanged
A      = H.flatten(2).softmax(-1)          # spatial attention -- unchanged
f_obj  = (F.flatten(2) * A).sum(-1)        # (B*L, D)      -- unchanged

q_obj  = self._presence_mlp(f_global).sigmoid() # (B*L, 1)  NEW: Linear(D,D/4)-ReLU-Linear(D/4,1)
f_obj  = q_obj * f_obj                      # trust gate

im_feat = f_global + self._gamma * self._obj_proj(f_obj) # residual fusion
# self._gamma = nn.Parameter(torch.zeros(1))           # step 0 == vanilla HiCE
# self._obj_proj = nn.Linear(D, D)

res['obj_heatmap'] = H                      # for heatmap loss + visualization
res['obj_presence'] = q_obj                 # for presence loss + q_obj accuracy metric
```

- Presence MLP input is **f_global only**. It is already hand-conditioned via cross-attention; surfacing `f_hand` out of `HandCrossAttention` adds wiring and missing-hand validity handling for little expected gain. Add `f_hand` later only if `q_obj` accuracy is poor.
- Heatmap head, softmax pooling, and the heatmap loss are unchanged from the reference PDF.



Red = changed vs previous plan (gate + fusion) · Teal = new scalar supervision · gamma initialized to 0

Object branch after the change. Red modules replace the old gate and fusion; the teal loss is the only new supervision. The residual path keeps f_{global} as the backbone of im_{feat} .

3. Supervision: presence target y_{obj}

The target comes for free from the rendered 7×7 grid already computed in frame.py — but do NOT use "any positive cell". With soft coverage targets, a 2%-visible sliver at the crop edge makes the grid technically nonzero while the pooled readout is garbage. Use a threshold with an ignore band:

```
# in frame.py, right after render_obj_target(...)
peak = obj_target.max()
if peak == 0:          y_obj, y_obj_valid = 0.0, 1.0    # nothing in crop
elif peak > TAU:      y_obj, y_obj_valid = 1.0, 1.0    # usable object visible
else:                 y_obj, y_obj_valid = 0.0, 0.0    # borderline sliver: mask loss
# TAU = 0.1 to start (max cell coverage threshold)
```

Loss terms (epoch()):

```
L_heatmap = masked per-cell BCE(H, obj_target)          # unchanged
L_presence = BCE(q_obj, y_obj) * y_obj_valid            # NEW, scalar per frame
L_obj      = L_heatmap + lambda_presence * L_presence   # lambda_presence = 0.5-1.0
L_total    = L_HiCE + lambda_obj * L_obj               # lambda_obj = 0.1-0.2
```

4. Mandatory data check before implementing the presence head

Compute the base rate of $y_{\text{obj}} = 0$ across the training crops. Annotations are dense and crops are hand-centered, so all-zero-target frames may be only a few percent. If negatives are that scarce, plain BCE teaches $q_{\text{obj}} \rightarrow 1$ always, the gate saturates, and the mechanism silently becomes a no-op (and the gate ablation row shows "no difference" for the wrong reason).

- If negative rate $< \sim 5\%$: weight the negative class in the presence BCE (pos_weight analogue), e.g. weight negatives by pos_count/neg_count.
- If negative rate is healthy ($> \sim 10\%$): plain BCE is fine.
- Either way, log q_{obj} accuracy separately on positive and negative frames during training — a gate stuck at 1.0 on negatives is the failure signature to watch.

5. Code diff surface (delta only)

File / location	Change	~Lines
model/model.py (init)	REMOVE: nothing shipped yet for the old gate if not written. ADD: <code>_presence_mlp</code> (Linear $D \rightarrow D/4$, ReLU, Linear $D/4 \rightarrow 1$), <code>_obj_proj</code> (Linear $D \rightarrow D$), <code>_gamma = nn.Parameter(zeros(1))</code> . REMOVE the concat fusion Linear($2D \rightarrow D$) if already written.	~10
model/model.py (forward)	Replace $\sigma(\max(H))$ gating and concat fusion with the NEW block in Section 2. Add <code>obj_presence</code> to res dict.	~8
dataset/frame.py	After <code>render_obj_target</code> : compute <code>y_obj</code> and <code>y_obj_valid</code> with TAU threshold + ignore band; add both to the batch dict.	~8
epoch()	Add <code>L_presence</code> (masked scalar BCE) into <code>L_obj</code> ; new <code>lambda_presence</code> arg; log presence accuracy split by pos/neg frames.	~10
main.py / args	<code>--lambda_presence</code> , <code>--presence_tau</code> ; keep <code>--obj_head</code> master flag.	~4

Everything else	Heatmap head, heatmap loss, target renderer, temporal stack, heads, NMS, eval — unchanged.	0
-----------------	--	---

6. Rollout order

- **Step 1 — prototype run:** keep $\sigma(\max(H))$ if it is already coded (one line, no new supervision) OR skip the gate entirely for the very first smoke run; residual fusion with $\gamma=0$ already protects the baseline. Goal: end-to-end pipeline runs, step-0 metrics match vanilla HiCE exactly. If they do not, the fusion wiring is wrong — fix before anything else.
- **Step 2 — data check:** compute y_{obj} base rate (Section 4) and pick TAU by eyeballing borderline crops.
- **Step 3 — swap in presence gate:** drop-in change at the same point in forward; nothing downstream moves. Train with L_{presence} enabled.
- **Step 4 — verify:** q_{obj} accuracy on pos/neg frames, pointing-game accuracy for H, then event mAP. The two auxiliary metrics tell you whether a bad mAP is a localization failure or a trust failure.

7. Updated ablation table

Residual $\gamma=0$ fusion is used in ALL rows (safety property, not a variant). Rows:

- 1. HiCE baseline ($--\text{obj_head}$ off, bit-exact).
- 2. + heatmap loss only, no fusion (deep supervision).
- 3. + heatmap attention pooling, no gate.
- 4. + $\sigma(\max(H))$ gate.
- 5. + supervised presence gate q_{obj} (full method).

Rows 3–5 isolate the gate contribution; report q_{obj} accuracy and pointing-game alongside mAP for rows 4–5. Note for the paper: if the y_{obj} negative base rate turns out very low, expect rows 4 and 5 to be close — say so explicitly rather than letting a reviewer discover it.